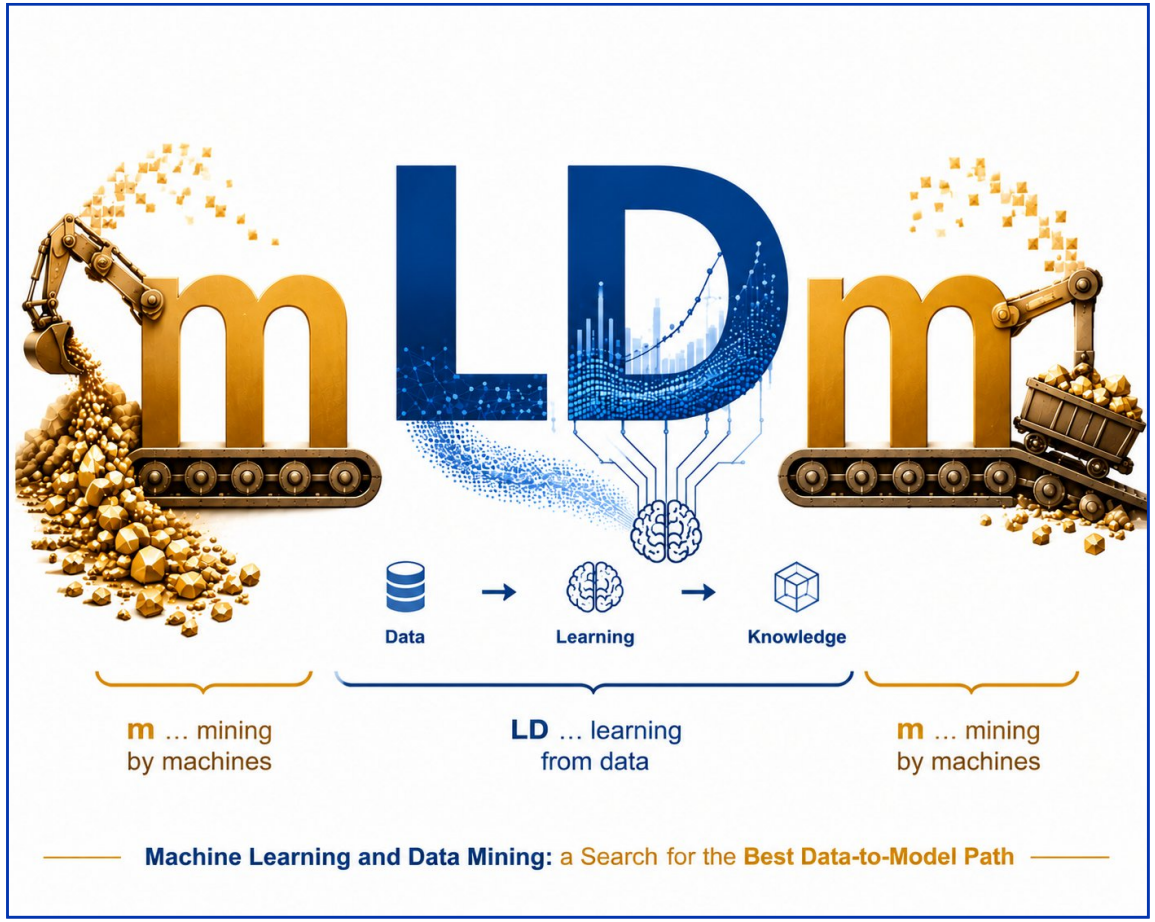
Technology to the Rescue

Paulo Trigo Silva

How to support the search for the best data-to-model path?



the overall process is not supported by a single tool

technological space to support a complete process requires a ***small ecosystem for,***

data, computation, experimentation, validation, deployment and reproducibility

Paulo Trigo Silva

Why technology matters?



- data must be stored, inspected, transformed and exported
- models must be trained, evaluated, persisted and used
- outputs must be submitted as evidence, not just described
- experiments must be reproducible and comparable

the chosen technology stack should support learning without hiding the process

The general technology space

field	typical technologies / capabilities
data acquisition	files, APIs, databases, logs, streams, sensors
storage and querying	relational DBMS, analytical DB, object storage
exploration and analytics	SQL, OLAP-views, notebooks, dashboards
transformation	python, SQL, pipelines, feature engineering libraries
modelling	machine learning libraries, model selection, metrics
reproducibility	virtual environments, containers, scripts, versioning
deployment	model persistence, APIs, batch scoring, dashboards
monitoring	logs, drift checks, operational validation, retraining triggers

what a complete life-cycle may require — not “all at once” but along the process

Paulo Trigo Silva

A controlled stack

(some) principles

- use open and “inspectable” technologies
- prefer explicit scripts and visible artifacts
- support progression from data-to-model

- the stack should be realistic enough to support the overall process life-cycle
- it should also remain simple enough to allow reasoning about each step
- must be defined before experimental work begins to reduce environment variability

The main components of the adopted stack

python `venv`
- isolated environment
- reproducible execution

workspace
- standard folder structure
- input-code-output-report

podman
- containerised services
- with open-source tooling

scikit-learn
- pipelines and models
- metrics and evaluation

postgreSQL
- relational source data
- SQL-based transforms

duckDB
- analytical queries
- file-based exploration

pgAdmin
- visual database inspect
- interactive SQL access

jupyter
- interactive experiments
- notebook evidence

...
- ...
- ...

not the whole technology space

a controlled subset

Paulo Trigo Silva

The workspace as a “technology” — structure is part of reproducibility

folder / artefact	role
`01_input`	provided inputs, templates and data
`02_code`	scripts, notebooks and code supplied or edited
`03_output`	generated outputs and automatic evidence
`04_report`	student-created evidence, notebooks or validation notes
`submission .zip`	controlled package with the evidence to submit

workspace

- a standard workspace avoids confusion between input, code, output and evidence
- also prepares later automation of experiments and submissions

Paulo Trigo Silva

Experimentation technologies — making data-to-model path executable

python

- general execution language
- scripts and automation

jupyter

- interactive reasoning
- narrative + code + evidence

scikit-learn

- pipelines and data-transforms
- learners and evaluation metrics

- **python** scripts support reproducible actions
- **jupyter** supports guided exploration/explanation
- **scikit-learn** supports the major learning tasks

Paulo Trigo Silva

Data technologies — from relational-sources to analytical datasets

postgreSQL

- object-relational source data
- schema, table, join, view

pgAdmin

- interactive data inspection
- visual database management

duckDB

- multi-dimensional queries
- lightweight analytics

- **postgreSQL** supports the idea that MLDM often starts from a (semi)structured data-source
- **pgAdmin** makes data layer more visible and easier to inspect and simplifies management
- **duckBD** supports analytical exploration that is more closely related to files and notebooks

Paulo Trigo Silva

Container technology — the “podman” approach

problem

services such as a DBMS are hard to install consistently on every machine

solution

run services within an isolated and reproducible environment

selected-option

podman and docker are available alternatives; podman is open-source

goal is not to explore container technology in depth; is to get a reproducible setting

Paulo Trigo Silva

The technology-to-process mapping



stage	technology support
formulate hypothesis	notebooks, SQL exploration, views
data analytics	duckDB, pandas
data transformation	postgreSQL, duckDB, pandas
feature representation	scikit-learn transformers, text/vector features (later)
model construction	scikit-learn, pipelines, metrics
operational validation	scripts, saved models, controlled test cases
deploy and operate	model persistence, prediction scripts, APIs (later)
monitor and improve	logs, repeated evaluation, drift checks (later)

each process stage (may) need a different technology support

The technology stack must produce evidence (artifacts) — the MoP

- each “**Module-of-Practice**” (**MoP**) aims to achieve some experimental goals
- the **MoP realization** should leave traceable and submitable **evidence** (artifacts)
- experimental work may use the proposed stack along with any other technologies
- additional technologies used in any experimental work must be properly explained

evidence type	**an example —** *using the* MoP *where the stack is initiated*
environment evidence	check scripts write results to `output` folder
notebook evidence	students create and save a small validation output
database evidence	services must be reachable and inspectable
submission evidence	`.zip` package contains controlled output folders

A (more) general technology space

field	typical technologies	*perspective*
data-stream	kafka, stream processors	conceptual (for now) later examples may discuss stream acquisition
dashboards	business intelligence (BI) OLAP tools	introduced as-a-tool through analytics and hyper-cubes
model registry	MLflow-like systems	not used conceptually linked to reproducibility
APIs	FastAPI, web services	deferred possibly later as deployment alternative
monitoring	drift dashboards, logs	conceptual (for now)
vector-databases	embedding stores	optional later for text / embeddings / retrieval contexts

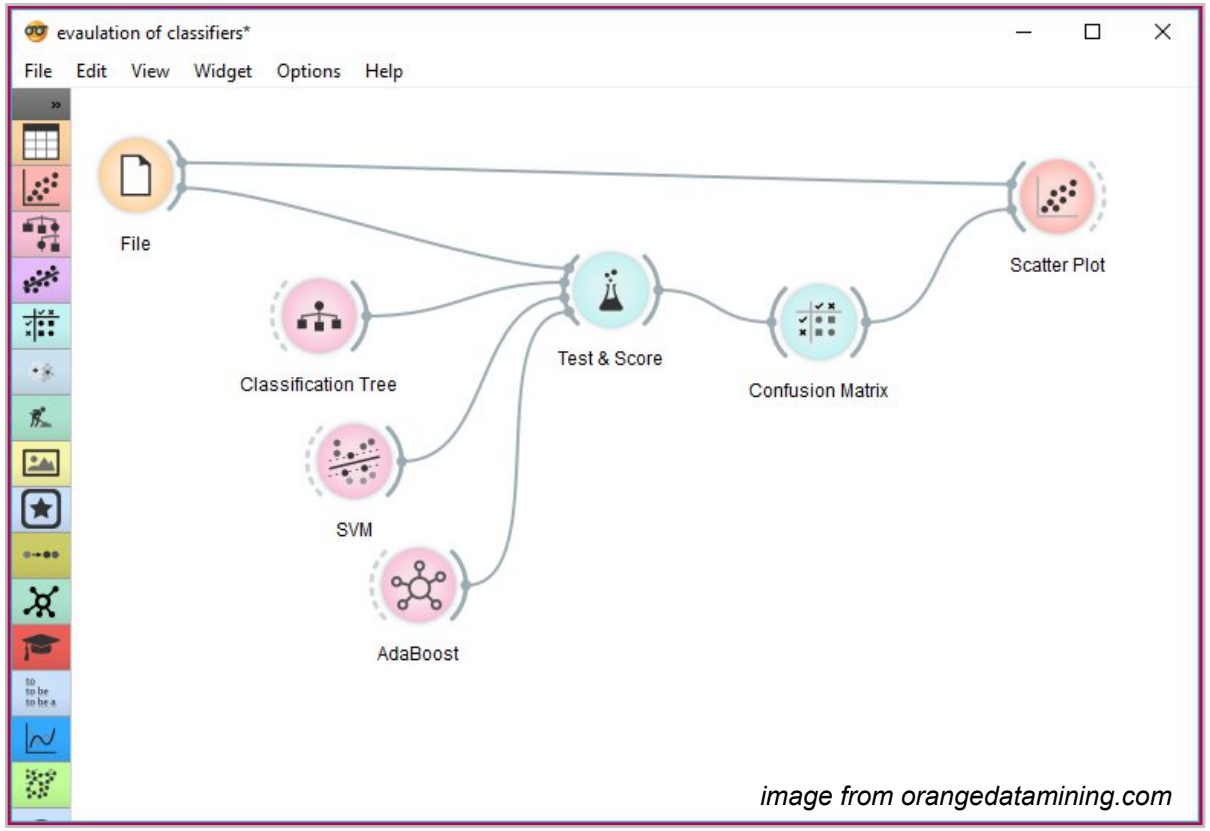
what we use now and what remains conceptually relevant; a “larger map”

The “visual” technology” — an additional (very relevant) tool

Orange Data-Mining (DM) Tool

- useful to quickly explore paths
- it lowers the “entry barrier”
- helps to keep a high-level view

- a path is a set of linked widgets
- focus is **on modeling** decisions
- focus is **not “on syntax”** or code
- use it as a “bridge” to other-tools



example workflow that explores some paths:
load data — compare classifiers — inspect errors

visual workflows to quickly and simply explore data-to-model path alternatives

Paulo Trigo Silva

A final “one sentence” message



the goal of the technology stack is not to add tools

*is to make the **search** for the best data-to-model path*

executable, inspectable, reproducible

and ready for an evidence-based experimental approach